

COMP2111 Week 4

Term 1, 2023

Propositional Logic

Summary of topics

- Well-formed formulas (SYNTAX)
- Boolean Algebras
- Valuations (SEMANTICS)
- CNF/DNF
- Proof
- Natural deduction

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction

Well-formed formulas

Let $\text{PROP} = \{p, q, r, \dots\}$ be a set of propositional letters.
Consider the alphabet

$$\Sigma = \text{PROP} \cup \{\top, \perp, \neg, \wedge, \vee, \rightarrow, \leftrightarrow, (,)\}.$$

The **well-formed formulas** (wffs) over PROP is the smallest set of words over Σ such that:

- $\top, \perp$ and all elements of PROP are wffs
- If φ is a wff then $\neg\varphi$ is a wff
- If φ and ψ are wffs then $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \rightarrow \psi)$, and $(\varphi \leftrightarrow \psi)$ are wffs.

Examples

The following are well-formed formulas:

- $(p \wedge \neg \top)$
- $\neg(p \wedge \neg \top)$
- $\neg\neg(p \wedge \neg \top)$

The following are **not** well-formed formulas:

- $p \wedge \wedge$
- $p \wedge \neg \top$
- $(p \wedge q \wedge r)$
- $\neg(\neg p)$

Conventions

To aid readability some conventions and binding rules can and will be used.

- Parentheses omitted if there is no ambiguity (e.g. $p \wedge q$)
- $\neg$ binds more tightly than $\wedge$ and $\vee$, which bind more tightly than $\rightarrow$ and $\leftrightarrow$ (e.g. $p \wedge q \rightarrow r$ instead of $((p \wedge q) \rightarrow r)$)

Conventions

To aid readability some conventions and binding rules can and will be used.

- Parentheses omitted if there is no ambiguity (e.g. $p \wedge q$)
- $\neg$ binds more tightly than $\wedge$ and $\vee$, which bind more tightly than $\rightarrow$ and $\leftrightarrow$ (e.g. $p \wedge q \rightarrow r$ instead of $((p \wedge q) \rightarrow r)$)

Other conventions (rarely used/assumed in this course):

- $'$ or $\bar{}$ for $\neg$
- $+$ for $\vee$
- $\cdot$ or juxtaposition for $\wedge$
- $\wedge$ binds more tightly than $\vee$
- $\wedge$ and $\vee$ associate to the left: $p \vee q \vee r$ instead of $((p \vee q) \vee r)$
- $\rightarrow$ and $\leftrightarrow$ associate to the right: $p \rightarrow q \rightarrow r$ instead of $(p \rightarrow (q \rightarrow r))$

Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

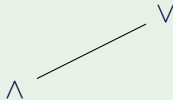
$\vee$

Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

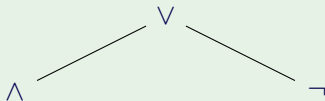


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

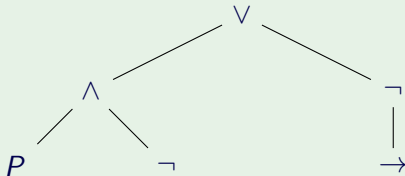


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$

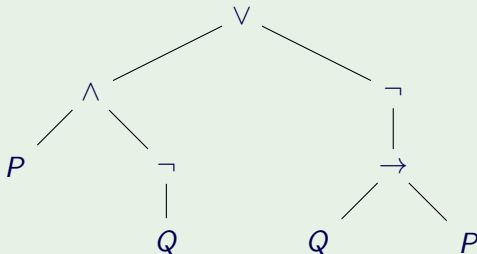


Parse trees

The structure of well-formed formulas (and other grammar-defined syntaxes) can be shown with a **parse tree**.

Example

$$((P \wedge \neg Q) \vee \neg(Q \rightarrow P))$$



Parse trees formally

Formally, we can define a parse tree as follows:

A parse tree is either:

- (B) A node containing $\top$;
- (B) A node containing $\perp$;
- (B) A node containing a propositional variable;
- (R) A node containing $\neg$ with a single parse tree child;
- (R) A node containing $\wedge$ with two parse tree children;
- (R) A node containing $\vee$ with two parse tree children;
- (R) A node containing $\rightarrow$ with two parse tree children; or
- (R) A node containing $\leftrightarrow$ with two parse tree children.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- CNF/DNF
- Proof
- Natural deduction

Laws (Week 2 flashback)

Look again at the laws of set operations and boolean equivalences:

$$x \cup y = y \cup x$$

$$x \cap y = y \cap x$$

$$(x \cup y) \cup z = x \cup (y \cup z)$$

$$(x \cap y) \cap z = x \cap (y \cap z)$$

$$x \cup (y \cap z) = (x \cup y) \cap (x \cup z)$$

$$x \cap (y \cup z) = (x \cap y) \cup (x \cap z)$$

$$x \cup \emptyset = x, \quad x \cap \mathbb{U} = x$$

$$x \cup x^c = \mathbb{U}, \quad x \cap x^c = \emptyset$$

$$x \vee y \equiv y \vee x$$

$$x \wedge y \equiv y \wedge x$$

$$(x \vee y) \vee z \equiv x \vee (y \vee z)$$

$$(x \wedge y) \wedge z \equiv x \wedge (y \wedge z)$$

$$x \vee (y \wedge z) \equiv (x \vee y) \wedge (x \vee z)$$

$$x \wedge (y \vee z) \equiv (x \wedge y) \vee (x \wedge z)$$

$$x \vee \perp \equiv x, \quad x \wedge \top \equiv x$$

$$x \vee \neg x \equiv \top, \quad x \wedge \neg x \equiv \perp$$

Laws (Week 2 flashback)

Look again at the laws of set operations and boolean equivalences:

$$x \cup y = y \cup x$$

$$x \cap y = y \cap x$$

$$(x \cup y) \cup z = x \cup (y \cup z)$$

$$(x \cap y) \cap z = x \cap (y \cap z)$$

$$x \cup (y \cap z) = (x \cup y) \cap (x \cup z)$$

$$x \cap (y \cup z) = (x \cap y) \cup (x \cap z)$$

$$x \cup \emptyset = x, \quad x \cap \mathbb{U} = x$$

$$x \cup x^c = \mathbb{U}, \quad x \cap x^c = \emptyset$$

$$x \vee y \equiv y \vee x$$

$$x \wedge y \equiv y \wedge x$$

$$(x \vee y) \vee z \equiv x \vee (y \vee z)$$

$$(x \wedge y) \wedge z \equiv x \wedge (y \wedge z)$$

$$x \vee (y \wedge z) \equiv (x \vee y) \wedge (x \vee z)$$

$$x \wedge (y \vee z) \equiv (x \wedge y) \vee (x \wedge z)$$

$$x \vee \perp \equiv x, \quad x \wedge \top \equiv x$$

$$x \vee \neg x \equiv \top, \quad x \wedge \neg x \equiv \perp$$

These are the same laws written with different symbols!

Definition: Boolean Algebra

A *Boolean algebra* is a structure $(T, \vee, \wedge, ', 0, 1)$ where

- $0, 1 \in T$
- $\vee : T \times T \rightarrow T$ (called **join**)
- $\wedge : T \times T \rightarrow T$ (called **meet**)
- $' : T \rightarrow T$ (called **complementation**)

and the following laws hold for all $x, y, z \in T$:

commutativity: $x \vee y = y \vee x$

$$x \wedge y = y \wedge x$$

associativity: $(x \vee y) \vee z = x \vee (y \vee z)$

$$(x \wedge y) \wedge z = x \wedge (y \wedge z)$$

distributivity: $x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$$

identity: $x \vee 0 = x, \quad x \wedge 1 = x$

complementation: $x \vee x' = 1, \quad x \wedge x' = 0$

Examples of Boolean Algebras

The set of subsets (power set) of a set $\mathcal{U}$:

- $T : \text{Pow}(\mathcal{U})$
- $\wedge : \cap$
- $\vee : \cup$
- $' : ^c$
- $0 : \emptyset$
- $1 : \mathcal{U}$

Examples of Boolean Algebras

The two element Boolean Algebra :

$$\mathbb{B} = (\{\text{true}, \text{false}\}, \&\&, ||, !, \text{false}, \text{true})$$

where $!$, $\&\&$, $||$ are defined as:

- $!\text{true} = \text{false}; !\text{false} = \text{true},$
- $\text{true} \&\& \text{true} = \text{true}; \dots$
- $\text{true} || \text{true} = \text{true}; \dots$

NB

We will often use $\mathbb{B}$ for the two element set $\{\text{true}, \text{false}\}$. For simplicity this may also be abbreviated as $\{T, F\}$ or $\{1, 0\}$.

Examples of Boolean Algebras

Cartesian products of $\mathbb{B}$, that is n -tuples of 0's and 1's with Boolean operations, e.g. $\mathbb{B}^4$:

$$\text{join: } (1, 0, 0, 1) \vee (1, 1, 0, 0) = (1, 1, 0, 1)$$

$$\text{meet: } (1, 0, 0, 1) \wedge (1, 1, 0, 0) = (1, 0, 0, 0)$$

$$\text{complement: } (1, 0, 0, 1)' = (0, 1, 1, 0)$$

$$0: (0, 0, 0, 0)$$

$$1: (1, 1, 1, 1).$$

NB

These are the bitwise operations on 4-bit machine words.

Examples of Boolean Algebras

Functions from any set S to $\mathbb{B}$; their set is denoted $\text{Map}(S, \mathbb{B})$

If $f, g : S \longrightarrow \mathbb{B}$ then

- $(f \vee g) : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto f(s) \parallel g(s)$
- $(f \wedge g) : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto f(s) \&\& g(s)$
- $f' : S \longrightarrow \mathbb{B}$ is defined by $s \mapsto !f(s)$
- $0 : S \longrightarrow \mathbb{B}$ is the function $f(s) = \text{false}$
- $1 : S \longrightarrow \mathbb{B}$ is the function $f(s) = \text{true}$

All those examples were just the power set example in disguise.

- The two-element algebra with $\{\text{true}, \text{false}\}$ is just the power set algebra with $\mathcal{U} = \{\emptyset\}$, but with different notation.
- A function $f : S \rightarrow \mathbb{B}$ coincides with the membership relation $(\in)$ of the set $\{x : x \in S \text{ and } f(x) = \text{true}\}$
- ...

All those examples were just the power set example in disguise.

- The two-element algebra with $\{\text{true}, \text{false}\}$ is just the power set algebra with $\mathcal{U} = \{\emptyset\}$, but with different notation.
- A function $f : S \rightarrow \mathbb{B}$ coincides with the membership relation $(\in)$ of the set $\{x : x \in S \text{ and } f(x) = \text{true}\}$
- ...

In fact, every boolean algebra is the power set example in disguise!

Every finite Boolean algebra satisfies: $|T| = 2^k$ for some k .

All algebras with the same number of elements are **isomorphic**, i.e. “structurally similar”, written $\simeq$. Therefore, studying one such algebra describes properties of all.

The algebras mentioned above are all of this form

- n -tuples $\simeq \mathbb{B}^n$
- $\text{Pow}(S) \simeq \mathbb{B}^{|S|}$
- $\text{Map}(S, \mathbb{B}) \simeq \mathbb{B}^{|S|}$

Duality revisited

If E is an expression made up with $\wedge, \vee, ', 0, 1$ and variables; then $\text{dual}(E)$ is the expression obtained by replacing $\wedge$ with $\vee$ and vice-versa; and 0 with 1 and vice-versa.

Theorem (Principle of Duality)

If $E_1 = E_2$ holds in all Boolean Algebras^a, then $\text{dual}(E_1) = \text{dual}(E_2)$ holds in all Boolean Algebras.

^ai.e. is provable using the Boolean Algebra Laws

In Week 2, we convinced ourselves of this with some handwaving.

Duality revisited

If $(T, \vee, \wedge, ', 0, 1)$ is a Boolean algebra, then the **dual algebra** $(T, \wedge, \vee, ', 1, 0)$ is also a Boolean Algebra. For example:

- $T : \text{Pow}(X)$
- $\wedge : \cup$
- $\vee : \cap$
- $' : ^c$
- $0 : X$
- $1 : \emptyset$

The principle of duality follows immediately from this observation.

Dualising, formally

A Boolean Algebra **expression** is defined inductively as follows:

- $0, 1$ are expressions
- A variable, $x, y, \dots$, is an expression.
- If E is an expression then E' is an expression.
- If E_1 and E_2 are expressions, then $(E_1 \wedge E_2)$ and $(E_1 \vee E_2)$ are expressions.

Use EXP for the set of such expressions.

Dualising, formally

We define $\text{dual} : \text{EXP} \rightarrow \text{EXP}$ recursively as follows:

- $\text{dual}(0) = 1, \text{dual}(1) = 0$
- $\text{dual}(x) = x$ for all variables x
- $\text{dual}(E') = \text{dual}(E)'$ for all expressions E
- $\text{dual}((E_1 \wedge E_2)) = (\text{dual}(E_1) \vee \text{dual}(E_2))$ for all expressions E_1 and E_2
- $\text{dual}((E_1 \vee E_2)) = (\text{dual}(E_1) \wedge \text{dual}(E_2))$ for all expressions E_1 and E_2

Example dual

$$\text{dual}((x \vee (x \wedge y))) = (\text{dual}(x) \wedge \text{dual}((x \wedge y)))$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y)))\end{aligned}$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge (\text{dual}(x) \vee \text{dual}(y)))\end{aligned}$$

Example dual

$$\begin{aligned}\text{dual}((x \vee (x \wedge y))) &= (\text{dual}(x) \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge \text{dual}((x \wedge y))) \\ &= (x \wedge (\text{dual}(x) \vee \text{dual}(y))) \\ &= (x \wedge (x \vee y))\end{aligned}$$

Boolean algebras: punchline

- Boolean algebras allow us to talk about set operations, propositions and bitwise operations on machine words at the same time.
- Properties proved in boolean algebra hold for all these instances: no more double labour.
- Calculation with set operations and calculation with propositional logic is, ultimately, the same thing. Therefore, you can freely change between the two perspectives in specifications and proofs.

Summary of topics

- Well-formed formulas
- Boolean Algebras
- **Valuations**
- CNF/DNF
- Proof
- Natural deduction

Valuations

A **truth assignment** (or **valuation**, or **model**) is a function

$$v : \text{PROP} \rightarrow \mathbb{B}$$

In week 2, we said:

*Two formulas ϕ, ψ are **logically equivalent**, denoted $\phi \equiv \psi$ if they have the same truth value for all truth valuations.*

But we never really defined valuation of formulas—instead, we handwaved it by saying “draw a truth table”.

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \|\ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = !\llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \rightarrow \psi) \rrbracket_v = !\llbracket \varphi \rrbracket_v \ || \ \llbracket \psi \rrbracket_v$

Valuations

Let $v : \text{PROP} \rightarrow \mathbb{B}$ be a valuation.

We can extend v to a function $\llbracket \cdot \rrbracket_v : \text{WFFs} \rightarrow \mathbb{B}$ recursively:

- $\llbracket \top \rrbracket_v = \text{true}$, $\llbracket \perp \rrbracket_v = \text{false}$
- $\llbracket p \rrbracket_v = v(p)$
- $\llbracket \neg \varphi \rrbracket_v = \neg \llbracket \varphi \rrbracket_v$
- $\llbracket (\varphi \wedge \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \&\& \ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \vee \psi) \rrbracket_v = \llbracket \varphi \rrbracket_v \ \|\ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \rightarrow \psi) \rrbracket_v = \neg \llbracket \varphi \rrbracket_v \ \|\ \llbracket \psi \rrbracket_v$
- $\llbracket (\varphi \leftrightarrow \psi) \rrbracket_v = (\neg \llbracket \varphi \rrbracket_v \ \|\ \llbracket \psi \rrbracket_v) \ \&\& \ (\neg \llbracket \psi \rrbracket_v \ \|\ \llbracket \varphi \rrbracket_v)$

Satisfiability, Validity and Equivalence

A formula φ is

- **satisfiable** if $\llbracket \varphi \rrbracket_v = \text{true}$ for some model v (v **satisfies** φ)
- **valid** or a **tautology** if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v
- **unsatisfiable** or a **contradiction** if $\llbracket \varphi \rrbracket_v = \text{false}$ for all models v

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ *is an equivalence relation.*

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ is an equivalence relation.

Example

- Commutativity: $(p \vee q) \equiv (q \vee p)$
- Double negation: $\neg\neg p \equiv p$
- Contrapositive: $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$
- De Morgan's: $(p \vee q)' \equiv p' \wedge q'$

Logical equivalence

Two formulas, φ and ψ , are **logically equivalent**, $\varphi \equiv \psi$, if $\llbracket \varphi \rrbracket_v = \llbracket \psi \rrbracket_v$ for all models v .

Theorem

$\equiv$ is an equivalence relation.

Example

- Commutativity: $(p \vee q) \equiv (q \vee p)$
- Double negation: $\neg\neg p \equiv p$
- Contrapositive: $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$
- De Morgan's: $(p \vee q)' \equiv p' \wedge q'$

Theorem

$\varphi \equiv \psi$ if, and only if, $(\varphi \leftrightarrow \psi)$ is a tautology.

Theories and entailment

A set of (well-formed) formulas is a **theory**

A model v *satisfies* a theory T if $\llbracket \varphi \rrbracket_v = \text{true}$ for all $\varphi \in T$

A theory T **entails** a formula φ , $T \models \varphi$, if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v which satisfy T

Example

- $T_1 = \{p\}$, $T_2 = \emptyset$, $T_3 = \{\perp\}$
- $v : p \mapsto \text{true}$ satisfies T_1 and T_2 but not T_3
- $T_1 \models (p \vee p)$ and $T_3 \models (p \vee p)$ but T_2 does not entail $(p \vee p)$

Theories and entailment

A set of (well-formed) formulas is a **theory**

A model v *satisfies* a theory T if $\llbracket \varphi \rrbracket_v = \text{true}$ for all $\varphi \in T$

A theory T **entails** a formula φ , $T \models \varphi$, if $\llbracket \varphi \rrbracket_v = \text{true}$ for all models v which satisfy T

Theorem

The following are equivalent:

- $\varphi_1, \varphi_2, \dots, \varphi_n \models \psi$
- $\emptyset \models ((\varphi_1 \wedge \varphi_2) \wedge \dots \varphi_n) \rightarrow \psi$
- $((\varphi_1 \wedge \varphi_2) \wedge \dots \varphi_n) \rightarrow \psi$ *is a tautology*
- $\emptyset \models \varphi_1 \rightarrow (\varphi_2 \rightarrow (\dots \rightarrow \varphi_n) \rightarrow \psi)) \dots$

Summary of topics

- Well-formed formulas
- Boolean Algebras
- Valuations
- **CNF/DNF** (tomorrow)
- Proof
- Natural deduction